Testing

Testing ::
It is a process of verifying any software (or) product is working as per expected behaviour or not

code :: Developer [Develops the product]

Testing :: Testers or QA Engineers (or) Test engineers, Storage QA

support :: Technical support Engineers

① Developers develop the software
② Testers tests the software (or) product
③ Technical support engineers will support the customers

Importance of Testing ::

→ Testing is performed to improve the quality of product

→ If we release the product without proper testing to the customers they are high chances of customers to find issues (defects / bugs)

→ If we test the product internally we can find the issues before customers and fix it that will help to improve the quality of product and overall company reputation

Different ways of Testing ::

① Manual Testing
② Automation Testing

① Manual Testing ::
It is the process of verifying product or software manually to check it is working as per expected behavior or not

(or)
Executing the test cases without depend on any tools

Advantages ::

① Easy to test (no need to write any scripts for testing)
② Best way to cover manual intervention testcases

Disadvantages ::

① It takes more time
② less efficiency
③ Not suitable for long run testcases

② Automation testing ::

It is a process of verifying the product or software by using scripts or tools to check it is working as per expected behaviour or not

(or)

Executing the testcases with the help of a tool or script

Advantages ::

① It is more efficient
② It takes less time [save time]
③ less manual intervention

Disadvantages ::

① 100% test coverage won't be code
② Implementation cost will be high

why? Testing ?

① To find issues in developers code / product functionality

How to test ?

① Go through the functional specification document
② End to End information / features of product
③ Develop test scenarios
③ Develop test cases

Testcase :: Step by Step approach to verify product / functionality is working as per expected behaviour or not

Ex:: verify Edge browser refresh functionality

Step-1 :: Launch Edge :: Browser should launch
Step-2 :: Navigate to refresh icon :: Refresh icon should be clikable
Step-3 :: click on refresh icon :: page should refresh

Test Scenarious:: Define high level functionality
Test case :: Step by Step Approach with expected result and Actual result

After executing the testcase we will get PASS / FAIL Result. when all the steps in a testcase working as per expected behaviour, we need to pass the testcase.

Methods of Testing::

① White Box testing
② Black Box testing
③ Grey Box testing

① White box testing::
→ It is a process of testing to find the issues or bugs in software code or developers code
→ Testers must have knowledge on developers code

Advantages::
→ It helps in optimizing developer code

Disadvantages::
→ Due to the requirement of skilled people [Skilled testers] the implementation costs will be increased.

② Black Box testing::
→ It is a process of testing to find the issues or bugs in software product functionality

Advantages::
Developer code is not required for the testers

Disadvantages::
Inefficient testing will be done

③ Grey Box testing::
→ It is a method of testing to find the issues in both developers code and product functionality
→ Testers should have knowledge on the both developers code and product functionality

Advantages::
→ It has a combined benifit of white box and blackbox testing
→ Efficient testing will be done [Test coverage will be more]

Disadvantages ::

→ Skilled engineers are required

→ It takes more time to complete

SDLC [Software Development life cycle] ::

1. Idea
   ↓
2. Design
   ↓
3. coding
   ↓
4. Testing
   ↓
5. Release
   ↓
6. support
   ↓
7. End of life

① Idea :: Any software product or application should start with idea

② Design :: once the idea is approved architects will design the product

③ coding :: In coding phase developers will develop the code

④ Testing :: In testing phase tester will do the end to end testing to make sure that product is working fine and it has zero% defects

⑤ Release :: once the testing is done marketing team or owners will release the product into market

⑥ support :: once the product is released into market support team will resolve the customer issues

⑦ End of Life [EOL] :: After some years product will reach end of life and no support will be provided

STLC ::

It stands for software testing life cycle

① Requirement Analysis

↓

② Test planning

↓

③ Test Design

↓

④ Test Environment setup

↓

⑤ Test Execution

↓

⑥ Test Reporting

① Requirement Analysis ::

In the requirement Analysis phase, testers analysis the customers and work with the developers to get the requirements which are testable

② Test planning ::

In the test planning phase all the planning about testing will be done

① what to test and what not to test
② what test strategy to be followed
③ what will be the test environment:
④ Resource required for testing
⑤ Test schedule / Test estimation

③ Test Design ::

In the design phase we will write the test cases either a manual or automation test cases

④ Test Environment setup ::

In this phase we will build the test bed or test configuration for executing test cases

⑤ Test Execution ::

In the phase we will execute the testcase/scripts on the testbeds

→ we will get pass/fail result after executing the testcase

fail :: If the testcase is failed we will fail the test and raise a bug

⑥ Test Reporting ::

we will report the test results in this phase

→ we will prepare test report and send to the customers or stake holders

→ we will prepare bug report [How many bugs/defects and raised and how many are fixed]

BUILD ::

It is a working software or firmware [Build is a developers code]

→ once developer develop the code for all functionalities they will release build to testers

→ we need to install build on top of our DUT/SUT

DUT :: Device under Test

SUT :: System under Test / Software under Test

→ Dev code (C/C++) will be converted to

for software :: .exe

for Hardware :: .bin

How Dev Release the BUILD

1. JIRA Tool [They will attach build in JIRA Tool]
2. Build servers
3. Jenkins
4. Shared folders
5. Via Email

When they will Release ?

→ whenever there is a change in the code

Ex:: Defect fix, New functionality

→ what bases we will pick imp testcases?
Based on change points and major functionalities

Get BUILD from Dev
↓
Install it on Testbed [DUT]
↓
Start Executing the testcases
↓
PASS / FAIL
↓
Defect to the dev [BUG]

BUG :: Deviation from Expected behaviour

If testcase fails ??
→ we need to debug the issue and find the root cause of failure
① Product Issue
② Script Issue
③ Setup Issue

Test plan → It belongs to particular release we will pull some testcases from test suite and create test plan
Master Testplan >> All the testcases >> Test suite

Test suite :: Test suite is a collection of testcases
Test cycle :: for every release we need to create test cycle and copy test cases required for testing from master testplan

Release1
Testcycle1
100 Testcases >> PASS | FAIL

Release2
Testcycle2
100 Testcases >> PASS | FAIL

→ whoever execute the testcases they have to update result in testcycle

→ Initially tests will be "unexecuted" state in JIRA tool we need to change status to "In progress" and execute the testcases
After execution we need to change status to "PASS" or "FAIL"

Fields of testcase / components of a testcase

① S.NO
② Testcase ID
③ Summary :: Verifying google chrome functionality
④ Steps
⑤ Expected Results
⑥ Actual Results

Ex::

S.NO	Testcase ID	Summary	Steps	Expected Result	Actual Result
1.	- TDS-123	- Test to verify gogle chrome home Page	- Turn on the PC	- PC should power on	-
			- Navigate to google chrome	- google chrome icon should be visible	
			- click on google chrome	- google chrome should open	
			- Type goog-le.com	- should display search bar	

→ If all the steps are working as per expected behaviour we will pass the testcase
→ If at least one step is not working as a expected behaviour we need to fail the testcase and raise the bug

How to write testcases ::

functionality verification document or Product verification document::

→ this document contains the in detail explanation of

each functionality of a product or software or application

→ Tester will write the testcases by referring this document

Test Scenarious ::

→ we will write the testcases from test scenario

→ Test scenarios can be derived from functionality verification documents

→ Test scenarios are created to make sure that every single feature covered on the testing and converted to testcases

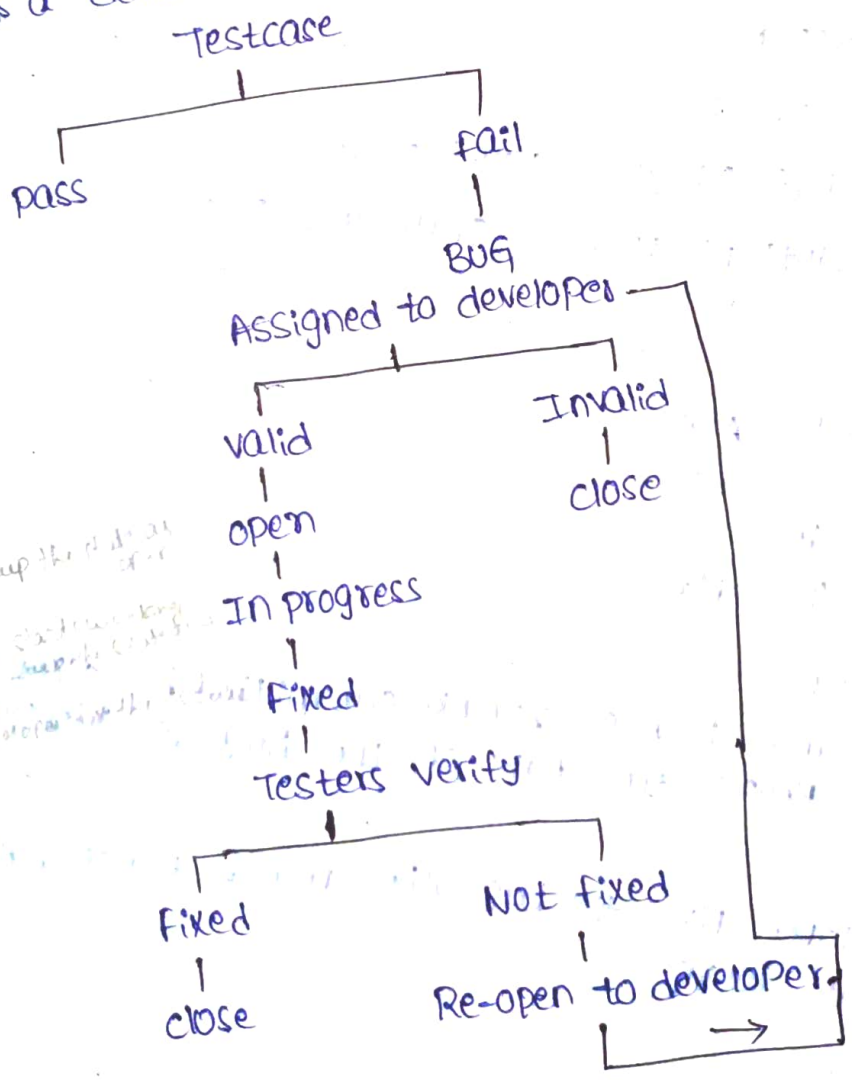
Ex ::

Test scenario :: verify google chrome functionality

Test case :: check refresh button functionality
check Home button functionality

BUG Life cycle ::

→ Bug is a deviation from expected behaviour



① After executing the testcase if it is failed testers will raise a bug

② As soon as tester raise a bug it will assigned to the developer and the state of bug will be new

③ once developer starts working on it He analyses it is valid or invalid bug. If it is valid bug he will keep the state as open

④ If it is invalid he will reject the bug and closes

⑤ If it is valid and once developer starts working on it, he will keep the state as inprogress

⑥ After developer fixes the issue, they will change the state of bug to "fixed" and release the build to testers

⑦ Testers will take the fix and verify that defects is really fixed or not

⑧ If it is working as per expected behaviour testers will close the defect

⑨ If it is not working as per expected behaviour testers will re-open the defect and re-assign it to the developer

States of BUG ::

New, open, Inprogress, fixed, close

BUG Resolutions :: final state of a BUG

close
fixed
Reject / false Alarm
Duplicate
cant Reproduce
Defered / wontfix

Duplicate ::

If the same bug is already submitted by other engineer, developer keeps the state of the bug as duplicate

Wont fix ::

If developer fixes the bug other major functionalities may not work

Deferred ::

If the developer is planned to fix the defect in next release and it is a valid defect they will make bug as deffered for the current release

Can't Reproduce ::

→ If the tester is not able to reproduce the issue (unable to hit the same issue on multiple test cases) then the developer resolve the issue by keeping resoultion as "can't reproduce"

BUG Tracking tools ::

① JIRA
② BUGZILLA
③ IMS
④ BURT [Netapp]
⑤ QC [HP]

Steps to file/ raise a BUG ::

① Log in to JIRA
② select project
③ click on create
   ⓐ select project [our project Name]
   ⓑ Issue type [BUG]
   ⓒ summary [one liner information about BUG]
   ⓓ Reporter [Test engineer name whoever reports the BUG]
   ⓔ Description [whatever steps followed to hit this issue] [issue reproduct]
   ⓕ Assignee :: [Developer] [Leave it to auto assignee]
   ⓖ Environment :: Testbed details [system configurations]
   ⓗ Attachment :: we need to attach the logs [Proof for the bug]. screenshots and system logs
   ⓘ priority ::
   ⓙ severity ::

script logs
System logs
dump logs

Priority :: — (developer)

How fast the developer needs to fix the issue

① Blocker
② Critical

③ Major
④ Minor
⑤ Trivial

Severity:: → (tester)
How severely the bug is impacting the functionality of a product
1.1 :: Highest
2.2 ::
3.3 ::
4.4 ::
5.5 :: Lowest

Functional testing ::
→ It is a type of software testing in which we will test the product/software against functional requirements are satisfied ~~by~~ ~~as~~ per the customer requirements or not
Ex :: FVT, unit testing, Smoke testing, Regression testing etc

Non-functional testing ::
→ It is a type of software testing in which we will test the behaviour of product/software under different workloads/circumstances
→ Here we will test on the aspects that are not covered in functional testing
Ex :: performance testing, load testing, stress testing

Different types of functional testing ::
① DVT [Design verification testing].
② FVT [functional verification testing/firmware Testing]
③ system testing
④ unit testing
⑤ Regression testing
⑥ Retesting
⑦ smoke testing
⑧ sanity testing

⑨ Adhoc testing
⑩ compatibility testing
⑪ Acceptance testing
Alpha testing
Beta testing

Different types of Non-functional testing ::
① performance testing
② security testing
③ scalability testing —

functional testing ::
② FVT [functional Verification Testing] ::
→ In FVT we will verify all the functionalities of a product are working as per expected behaviour or not
→ To make sure that all the functionalities are covered as per the requirements specifications we perform functional testing
(or)
→ Testing every possible option in the product is working as fine or not is called as FVT.

③ System Testing ::
→ It is a level of software testing in which we will test the "end to end" features of a product as whole with respect to the customer use cases
→ How the customer user out product we will execute those kind of scenarios
→ we need to wait for the complete product to develop system testing → Incase of functional testing we can test individual features (compl
⑤ Regression Testing :: → Executing the passed testcases with new build is called as Regression testing
product is not requir

Why ::
→ To make sure that other functionalities are not affected due to code changes

When we will regression testing ::
→ whenever there are code changes [New Build] we will do regression testing

1000 testcases
↓
Executed 1000 testcases
↓
700 pass
300 fail
↓
Raise a BUG
↓
Developer fixes and gives New Build

Execute 1000 testcases
↓
800 pass
200 fail
↓
Raise a BUG
↓
Developer fixes and gives new build

⑥ Retesting ::

→ Executing the failed testcases with new build to make sure that BUG's are fixed is called as re-testing

→ Retesting is also called as defect/BUG/JIRA verific-ation

→

④ Unit testing ::

→ It is also called as code level testing

→ In which developers will divide the source code into units and tests that is working or not before releasing the code [Build] to the testers

→ It will be done by developers

→ once unit testing passed only developers will release the build testers

Integration testing ::

→ It is a level of software testing which individual units are combined together and tested as a group

→ Integration starts with unit testing

→ Either developers or testers can perform integration testing

Non-functional testing ::

Performance Testing ::

→ It is a level of software testing in which we will test the performance of a device under test [DUT] with different workloads

They are two types of performance testing

① load testing

② stress testing

① load testing :: It is one of the method of performance testing in which we will keep the load within the supported level and check how the product / system / software / application is performing

② stress testing :: It is one of the type of performance testing in which we will keep the load above the supported level and check how the product / system / software / application is performing

functional testing ::

⑦ smoke testing ::

→ It is a level of software testing in which we will check the stability of a software / product [Built] in initial stage

→ It will be done by testers

→ It is also called as build verification testing

→ Here we will check build verification installation is successful or not

→ It will help us to find if any issues in the build in initial stage only

→ As soon as build is released into testers the first testing they do is "smoke testing"

⑧ sanity testing ::

→ whenever there is a less time we will pick the important testcase and execute. This process is called sanity testing

→ sanity testing will be done by testers

100 testcases

21 days

→ we will do complete regression testing

100 testcases
10 days
we pick some important testcases and perform sanity testing

Important features of a product
How to pick testcases → based on change points
① Basic functionality of a product
② features around the BUG fixes

⑩ compatibility testing::
→ In compatibility testing we will test the product compatibility with different OEM's (original Equipment Manufacturer)
Ex:: Intel CPU → processor
HP LAPTOP
DELL LAPTOP
LENOVO LAPTOP >> OEM

⑨ Adhoc Testing::
→ If we complete the assigned testcases early then we will do adhoc testing
→ Ex:: If we assign with 100 testcases to complete in 10 days but if we complete it in 9 days we will do adhoc testing
→ Adhoc testing is also called as unplanned testing
→ when testing is performed without any proper planning and documentation is called as adhoc testing
→ After completing the formal testing on the product testers will do adhoc testing

⑪ Acceptance Testing::
Basic Acceptance Testing [BAT] (or) user Acceptance Testing [UAT]
→ It is a level of testing in which we will test the product/software for acceptance

→ customer checks all their requirements are satisfied or not

Alpha Testing ::

→ customer come to our environment and execute their testcases

→ we execute the customer scenarious on top of our testbeds

Beta Testing ::

→ customer takes the product to their environment and executes their testcases

→ customers will deploy real time workloads on the our product

Monkey Testing ::

→ If the testing is performed without any knowledge on the product is called as Monkey Testing

Entry Criteria ::

→ Entry criteria tells us when to start the testing

→ Successful Build installation is an entry criteria for our testing

Exit Criteria ::

→ Exit criteria tellsus when to stop the testing

→ once the product is bug free we can stop the testing

Build 1 :: 1000 testcases - 100 Bugs
Build 2 :: " " - 50 Bugs
Build 3 :: " " - 5 Bugs
Build 4 :: " " - 0 Bugs → Exit criteria

→ once all the bugs are fixed, our product has met exit criteria.

① High priority and High severity
Ex:: payment transactions failing for users
Security breach - exposing customer data
production website is down - users can't access application

② Low priority and low severity
spelling or Typo error
Incorrect support email or website
Icon mismatch

③ High priority and low severity
wrong company name / logo shown on homepage
misleading error / success message
Notification sent with wrong content
Broken or wrong links

④ Low priority and High severity
Rare feature crashes the app
security issue in disabled feature
failure in admin only function

Test case writing Techniques ::

① Equivalence partition Technique
② Boundary value Analysis
③ Error Guessing

① Equivalence partition Technique ::

→ In equivalence partition Technique we will divede the input into equivalent partition and derive the testcases

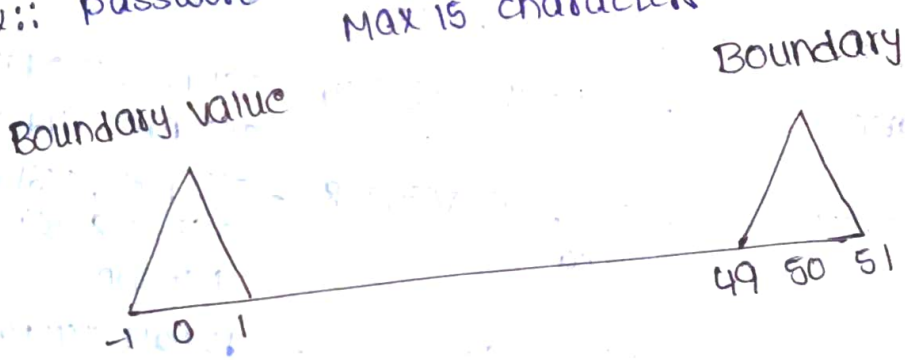
Ex:: Gmail Password
Minimum characters :: 8
Maximum characters :: 15

Less than Range	In Range	Above the Range
partition -1 less than 8-characters	Partition-2 valid 8-15 characters	Partition-3 Above 15 char -acters

② Boundary value Analysis ::

→ In Boundary value Analysis we will take the boundary value of supported inputs and we will derive the testcases

Ex:: password :: Min 8 characters
Max 15 characters



③ Error Guessing ::

→ In this method based on the prior experience on testing testers will guess the errors and write the testcases

Ex::

→ If the tester is already worked on RAID feature in their previous company / project they can guess that R5 creation will fail with minimum of 2 drives and write the testcases accordingly

Positive Testing ::

→ It is a type of testing in which we will provide the valid inputs and check how the product is working

Negative Testing ::

→ It is a type of testing in which we will provide invalid inputs and check how the product is working

the purpose of negative testing is our product [OUT] should not fail /crash and remains stable with the valid inputs

Traceability Matrix ::

→ this is also called as requirement traceability Matrix [RTM]

→ the relation b/w the testcases and the requirements are shown in a document is called as traceability Matrix

→ If testcase fails this document will help us to find the corresponding functionality easily

→ It also helps us to ensure that all the requirements are covered which are needed

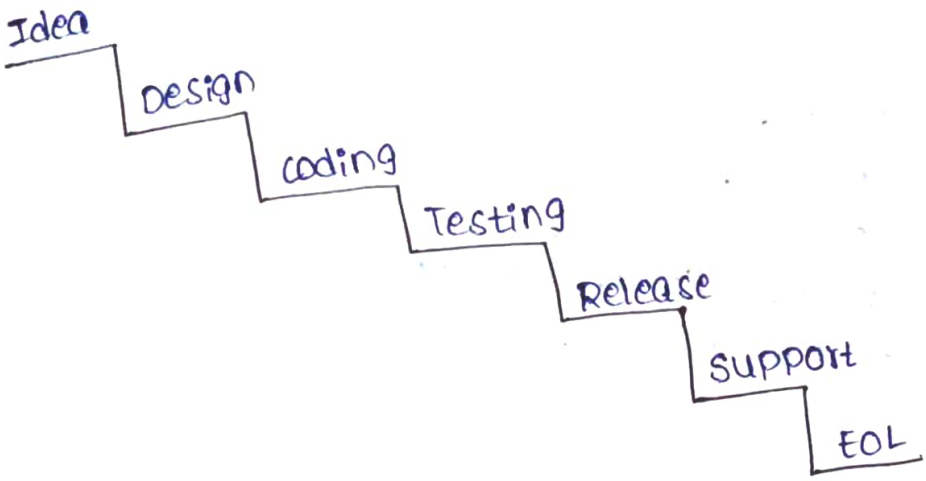
Ex ::

S.No	Requirement Description	Testcase ID	status
1.	Test Scenario 1	- TC1, TC2, TC3, TC4	- TC1 pass, TC2 fail, TC3 pass, TC4 fail
2.	Test Scenario 2	- TC5, TC6, TC7, TC8, TC9	- TC5 pass, TC6 fail, TC7, TC8, TC9 Not run
3.	Test Scenario 3	- TC10, TC11, TC12	- TC10 pass, TC11 pass, TC12 fail
4.	Test Scenario 4	- TC13, TC14, TC15	- TC13 pass, TC14 fail, TC15 Not run

Models of Testing::

① waterfall Model
② V-Model
③ Agile Model

① waterfall Model::



→ It is a step by step approach of following software development life cycle
→ After one step completes only, we can proceed to next step

Advantages::

① Easy to understand
② Each step is well planned
③ proper hardware and takeover happen
④ well suited for small projects

Disadvantages::

① Not suitable for big projects
② we can't change the design once it is moved to next phase
③ It consumes more time

③ Agile model::

→ the software testing that follows the principles of agile software development is called as Agile model
→ Agile is an interactive development where requirements involve through collaboration blw the customers and organizations [who develops the product or software]

→ Here coding and testing happens parallely

Components or features of Agile (or) principles of Agile::

1. sprint.
2. Scrum.
3. user Story - Multipe sprints, subtask.
4. Subtask
5. Backlog.
6. Story Points.
7. Scrum Master.
8. Sprint Review Meeting.
9. Scrum Meeting [Daily stand-up meeting].
10. Retrospective Meeting.

Epic::

→ Epic will be created once in a project lifecycle or once in a year

① sprint ::

→ sprint

scrum::

→ Scrum is a framework {set of rules & standards that we follow to implement Agile process (SDLC & STLC)

scrum master::

→ Is a person within the team who is responsible for creating sprints and conducting scrum meeting [Daily stand-up meeting] and discussing with the stake holders [customers or high-level employees]

→ He/she also responsible for creating user stories and assign it to all the engineers

→

Sprint ::

→ Sprint is a duration of time to complete assigned task

→ Sprint duration will be 2 to 3 weeks

→ scrum master is responsible for creating and managing sprint [creating user story and subtask]

week1 - week2 :: sprint 1
week3 - week4 :: sprint 2

→ In a year 24 sprints will release

...

week51 - week52 :: sprint 26 : Last sprint

Scrum meeting ::

→ It is also called as daily stand-up meeting

→ In which we will discuss about the status of assigned work and if any issues / blockers that are stopping our test execution

→ Developers and testers will have seperate scrum meetings

→ scrum master conducts the scrum meeting

→ All the team members will be part of meeting [Including manager, team lead, scrum master]

Sprint Review meeting or Sprint planning ::

→ this meeting happens before starting the sprint in which scrum master, Product owner / team lead manager will join and discuss about the sprint plan [what work to be assigned to each engineer]

User Story ::

→ It is a template where it contains the type of work that is assign to team

→ In a sprint if scrum master assigns the same work they will create a story for each type of work

Ex :: User story 1 :: Execute 20 testcases.
User Story 2 :: write automation scripts for 10 testcases

user story 3 :: Go through the product verification documents.

Subtask ::

→ subtask is a work that is part of user story, Each engineer will create subtask for their assigned work.

Ex:: In a 2weeks sprint [80 hours].

Subtask 1 :: 30 hours executing testcases.
Subtask 2 :: 30 hours writing automation scripts.
subtask 3 :: 10 hours meeting.
subtask 4 :: 10 hours going through the documents.

Story Points ::

→ for each user story scrum master / manager assigns the story points depends on the priority / importance of the particular work.

Backlog ::

→ Backlog is a remaining work from the current sprint that can be carried to next sprint

Retrospective Meeting ::

→ this meeting happens after the sprint [Last time of sprint]

→ In which we will be discuses how the previous sprint has happend [what went right and what went wrong]

→ And if there are any improvements are needed for planning the next sprint

Process in Agile Model :: To create New sprint

1. Accept All user Stories
2. create subtask
3. start subtask
4. update subtask
5. close subtask
6. close user story